



Bookly Pitch Deck

Proposed agenda

1. Background and architecture
2. Quick Demo
3. Deep dive into key design decisions
4. Decagon team grills Balram 🔥🔥🔥

How a Great CS Agent Should Work

The LLM drives the user experience, while the code and policy drive the decisions

Customers get a natural, conversational experience - while every important action, such as those relating to money, data, or trust is enforced, outside the model's judgment.

Through a 3 agent approach, Bookly can effectively resolve their most important customer support issues



Orders

Status, tracking, cancellation, address changes - resolved instantly, no human needed.



Returns & Refunds

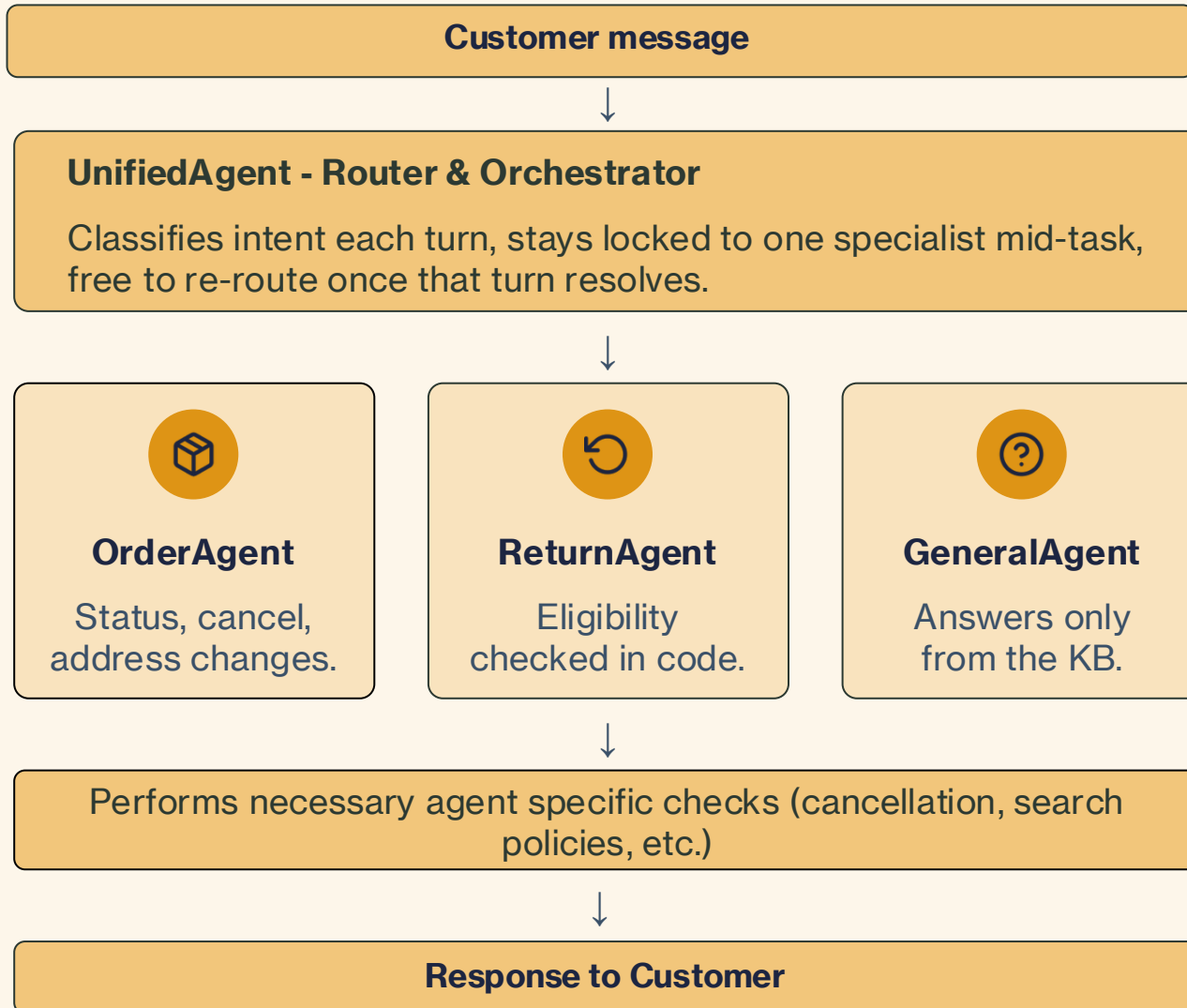
Collects the details, checks eligibility in code, hands off to a human for the final refund call.



General Q&A

Answers grounded strictly in Bookly's policy knowledge base - never a guess.

Architecture - How a Message Flows Through the System



1

Orchestration & Routing

One router classifies each message and hands it to the right specialist

2

Tools Layer

Every policy lives here as plain Python functions - eligibility, verification, cancellation - called once the agent is run.

3

Memory

Per-session state tracks what's been collected; shared 'last order discussed' lets follow-ups skip re-asking; chats are stored in history tab

4

NLU & Prompts

Regex for fixed syntax (IDs, emails, dates); one Claude call per turn for fuzzy meaning; KB-grounded answers, with an offline fallback.

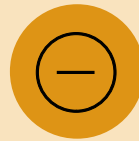
Key Decision 1:

Guardrails live in the code, not the Prompt



What we chose

- Every policy check is a plain Python function
- The agent calls the functions and obeys the result – it does not improvise
- If there is know knowledge base match, the agent will not guess



What we traded off

- Less flexibility than letting the model reason in natural language.
- Every new rule is a code change and a redeploy
- Slower to iterate on than adjusting wording.



Why it was worth it

- A prompt is a suggestion, while a function is a guarantee
- Not compromising anything touching money or trust
- Fully auditable in case of issues

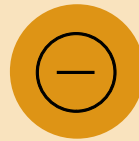
Key Decision 2:

Clarify ambiguity and require confirmation before taking any action



What we chose

- Ask when ambiguous; which order, which item , a vague reason, etc
- Require an explicit yes/no before any write actions: cancel, address change, refund handoff



What we traded off

- A few extra turns before the agent actions
- Longer customer experience from question to action



Why it was worth it

- A wrong guess on a destruction action or financial action costs more than a question
- Confirming before acting builds trust – both with the customers and the full organization

Next Steps with More Time



Top Priority: Tiered Auto-Refund Agent

Today every return escalates to a human for the refund amount. With more time, I'd let the agent resolve small, low-risk refunds itself. This would require thorough testing to make sure it has captured all edge cases.

The following Guardrails would be put in place and require escalation, regardless of policy:

**> 3 orders refunded
per each day**

**Subjective or
questionable reason
for return**

**Order greater than
\$25**

Other things we'd prioritize next



Durable storage

Move session & order state from in-memory dicts to a real database.



Real authentication

Replace order-number + email matching with authenticated sessions.



Evaluation harness

Automated guardrail regression tests & adversarial prompts on every change.